

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

Facultad de Ciencias de la Computación

Programación Concurrente y Paralela

Tarea 1

Modelado de soluciones paralelas

Equipo:

Islas Gómez Luis Antonio — 202372860
Ramírez Candia Alfredo — 202375947
Pérez Flores Julio César — 202247445
Navarro Soto Mario Alberto — 202264602

Docente: Hilda Castillo Zacatelco

21 de septiembre de 2026

Índice general

1	Introducción y objetivo	2
2	Marco teórico	2
3	Problema 1. Ordenamiento lexicográfico	2
4	Problema 2. Inversión de un arreglo	4
5	Problema 3. Intersección de conjuntos	5
6	Problema 4. Raíces reales de una ecuación cuadrática	8
7	Problema 5. Máximo de una matriz	9
8	Problema 6. Alumnos aprobados en matemáticas	11
9	Problema 7. Descuento en medicamentos	13
10	Conclusiones	15

1 Introducción y objetivo

El objetivo de esta tarea es modelar una solución paralela para cada uno de los ejercicios planteados, empleando tres herramientas de análisis: los **grafos de dependencia**, las sentencias **COBEGIN–COEND** y las **condiciones de Bernstein**. El modelado no consiste en repartir el trabajo “a ojo”, sino en identificar qué tareas pueden ejecutarse simultáneamente y cuáles deben respetar un orden obligatorio.

2 Marco teórico

- **COBEGIN–COEND:** notación de pseudocódigo que expresa paralelismo (fork/join). Las sentencias encerradas entre **COBEGIN** y **COEND** son concurrentes: no existe un orden predefinido entre ellas, y el proceso padre espera a que *todas* terminen antes de continuar después del **COEND**.
- **Condiciones de Bernstein:** dos bloques S_i y S_j , con $L(S)$ el conjunto de variables que *leen* y $E(S)$ el conjunto de variables que *escriben*, pueden ejecutarse en paralelo si y solo si se cumplen las tres condiciones:

$$L(S_i) \cap E(S_j) = \emptyset, \quad E(S_i) \cap L(S_j) = \emptyset, \quad E(S_i) \cap E(S_j) = \emptyset.$$

Si se cumplen, los bloques son *independientes* y van en el mismo **COBEGIN**; si alguna falla, existe una dependencia y deben colocarse en el orden que esta impone.

- **Grafo de dependencias:** los nodos son las tareas y una arista $A \rightarrow B$ indica que B no puede comenzar hasta que A termine. Los tipos clásicos son RAW (dependencia verdadera: escribir y luego leer), WAR (antidependencia: leer y luego escribir) y WAW (dependencia de salida: escribir y luego escribir). Los nodos sin aristas entre sí forman el paralelismo disponible; el camino más largo (camino crítico) fija el tiempo mínimo y el ancho del grafo el grado de paralelismo.

3 Problema 1. Ordenamiento lexicográfico

Enunciado

Ordenar lexicográficamente los elementos de un arreglo de cadenas de forma ascendente.

Análisis del algoritmo

Se emplea el ordenamiento por transposición par-impar. En cada fase se comparan e intercambian parejas de elementos adyacentes, alternando el índice de inicio:

$$\text{fase par: } S_i = \text{cmp-exch}(i, i + 1) \quad \text{para } i = 0, 2, 4, \dots, N - 2,$$

$$\text{fase impar: } S_i = \text{cmp-exch}(i, i + 1) \quad \text{para } i = 1, 3, 5, \dots, N - 2.$$

Dentro de una misma fase cada pareja toca índices disjuntos, por lo que las tareas son independientes; entre fases existe una dependencia porque una pareja puede leer un elemento que la fase anterior escribió.

Condiciones de Bernstein

Para cada tarea se identifican las variables que lee (L) y las que escribe (E):

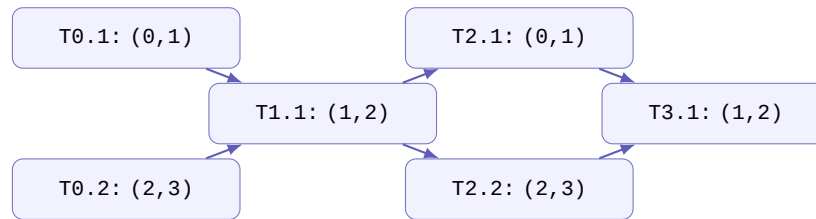
Tarea	Lee L	Escribe E
T1: <code>cmp-exch(A[i], A[i+1])</code>	$\{A[i], A[i+1]\}$	$\{A[i], A[i+1]\}$
T2: <code>cmp-exch(A[i+2], A[i+3])</code>	$\{A[i+2], A[i+3]\}$	$\{A[i+2], A[i+3]\}$

Verificación de independencia:

- **T1 || T2:** $L(T1) \cap E(T2) = \emptyset$, $E(T1) \cap L(T2) = \emptyset$ y $E(T1) \cap E(T2) = \emptyset$. Operan sobre índices disjuntos del arreglo, por lo que las tres condiciones se cumplen y son independientes dentro de la misma fase.
- **Fase $k \rightarrow$ fase $k+1$:** existe una dependencia RAW porque una tarea de la fase siguiente lee un elemento que una tarea de la fase actual escribió. Deben respetar ese orden mediante la barrera COEND.

Grafo de dependencias

Para que el grafo sea claro se modela el caso $N = 4$, que generaliza la estructura para cualquier N . Cada nodo $T_{f,i}$ es la comparación e intercambio de la pareja $(i, i+1)$ en la fase f .



El camino crítico es $T0.1 \rightarrow T1.1 \rightarrow T2.1 \rightarrow T3.1$ y el paralelismo máximo es $N/2$ en cada fase.

Solución con COBEGIN–COEND

```

1 para fase = 0 .. N-1 hacer
2   si fase es par entonces
3     COBEGIN
4       para i = 0, 2, 4, ..., N-2 hacer
5         cmp-exch(A[i], A[i+1])
6     fin para

```

```

7      COEND
8  sino
9      COBEGIN
10     para i = 1, 3, 5, ..., N-2 hacer
11         cmp-exch(A[i], A[i+1])
12     fin para
13 COEND
14 fin si
15 fin para

```

Todas las parejas de una misma fase son independientes, por lo que van dentro del mismo **COBEGIN**. La barrera **COEND** garantiza que ninguna tarea de la fase siguiente comience hasta que todas las de la fase actual hayan terminado, respetando la dependencia RAW entre fases.

4 Problema 2. Inversión de un arreglo

Enunciado

Dado un conjunto de números en un arreglo, invertir el arreglo, es decir, el primer elemento deberá aparecer al final, el último al inicio, y así sucesivamente.

Análisis del algoritmo

Sea A un arreglo de N elementos indexado de 0 a $N-1$. La inversión intercambia cada elemento con su simétrico respecto al centro:

$$\text{swap}(A[i], A[N-1-i]) \quad \text{para } i = 0, 1, \dots, \lfloor N/2 \rfloor - 1.$$

Cada tarea toca únicamente la pareja de posiciones $\{i, N-1-i\}$. Como los índices de una pareja pertenecen a la mitad inferior y superior respectivamente, los conjuntos de posiciones de dos parejas distintas son disjuntos, de modo que todas las tareas son independientes.

Condiciones de Bernstein

Para cada tarea se identifican las variables que lee (L) y las que escribe (E):

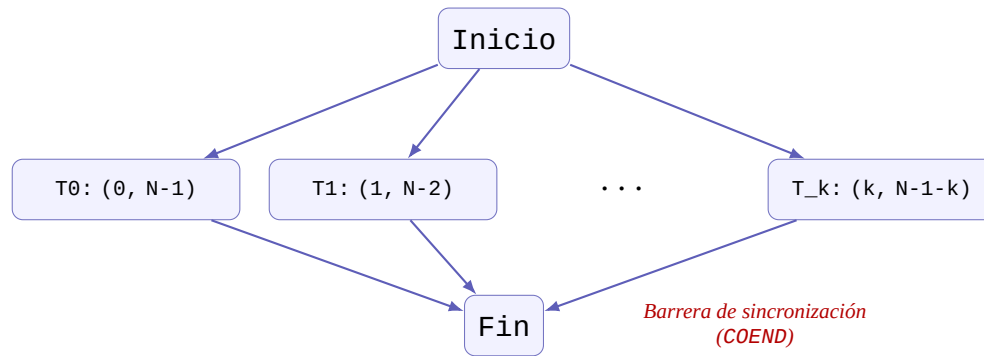
Tarea	Lee L	Escribe E
$T_i: \text{swap}(A[i], A[N-1-i])$	$\{A[i], A[N-1-i]\}$	$\{A[i], A[N-1-i]\}$
$T_j: \text{swap}(A[j], A[N-1-j])$	$\{A[j], A[N-1-j]\}$	$\{A[j], A[N-1-j]\}$

Verificación de independencia:

- $T_i \parallel T_j$ ($i \neq j$): como $\{i, N-1-i\} \cap \{j, N-1-j\} = \emptyset$, se cumple que $L(T_i) \cap E(T_j) = \emptyset$, $E(T_i) \cap L(T_j) = \emptyset$ y $E(T_i) \cap E(T_j) = \emptyset$. Las tareas son independientes y pueden ejecutarse en paralelo.
- **Inicio** $\rightarrow T_i \rightarrow$ **Fin**: no hay dependencias entre las tareas de intercambio; la única sincronización necesaria es la barrera **COEND** que separa el cálculo del resto del programa.

Grafo de dependencias

Las tareas de intercambio no tienen aristas entre sí: parten de un nodo común y convergen en una barrera de sincronización. Cada nodo T_i representa el intercambio de la pareja de posiciones $(i, N - 1 - i)$.



El camino crítico es $\text{Inicio} \rightarrow T_i \rightarrow \text{Fin}$ y el paralelismo máximo es $\lfloor N/2 \rfloor$.

Solución con COBEGIN–COEND

```
1 leer A[0..N-1]
2 COBEGIN
3   para i = 0 .. floor(N/2)-1 hacer
4     temp = A[i]
5     A[i] = A[N-1-i]
6     A[N-1-i] = temp
7   fin para
8 COEND
9 escribir A
```

Cada iteración del bucle intercambia una pareja distinta de posiciones, por lo que todas son independientes y van dentro del mismo COBEGIN. La barrera COEND garantiza que el arreglo esté completamente invertido antes de imprimirlo.

5 Problema 3. Intersección de conjuntos

Enunciado

Sean dos conjuntos de cadenas (sin duplicados internos):

$$A = \{a_1, a_2, \dots, a_n\}, \quad B = \{b_1, b_2, \dots, b_m\}$$

Determinar:

$$C = A \cap B = \{x \mid x \in A \wedge x \in B\}.$$

Solución secuencial clásica con costo $O(n \cdot m \cdot L)$, donde L es el costo de comparar dos cadenas.

Análisis del algoritmo

Para evitar conflictos de escritura sobre C el cálculo se descompone de la siguiente forma:

$$M[i, j] = (A[i] == B[j]), \quad P[i] = \bigvee_{j=1}^m M[i, j], \quad C = \{a_i \mid P[i] = 1\}.$$

1. **Comparación:** $T_{i,j}$ calcula $M[i, j] = 1$ si $a_i = b_j$, 0 en otro caso, con $1 \leq i \leq n$, $1 \leq j \leq m$.
2. **Reducción por filas:** S_i calcula $P[i] \in \{0, 1\}$. $P[i] = 1$ ssi $a_i \in B$.
3. **Compactación:** construcción de C a partir de P .

Condiciones de Bernstein

Para cada tarea se identifican las variables que lee (L) y las que escribe (E):

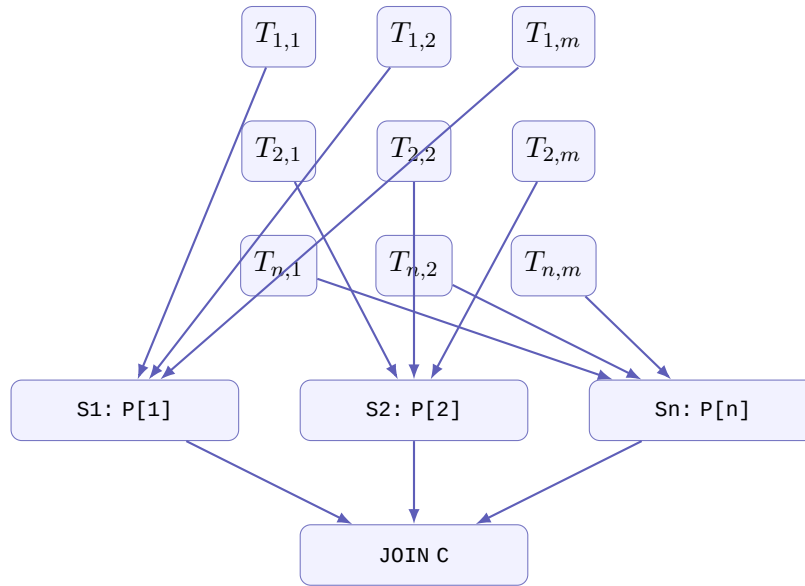
Tarea	Lee L	Escribe E
$T_{i,j}: M[i, j] = (A[i] == B[j])$	$\{A[i], B[j]\}$	$\{M[i, j]\}$
$S_i: P[i] = \text{OR}(M[i, :])$	$\{M[i, 1..m]\}$	$\{P[i]\}$
$\text{JOIN}: C = \{a_i \mid P[i]=1\}$	$\{P, A\}$	$\{C\}$

Verificación de independencia:

- $T_{i,j} \parallel T_{i',j'}: L(T_{i,j}) \cap E(T_{i',j'}) = \emptyset, E(T_{i,j}) \cap L(T_{i',j'}) = \emptyset$ y $E(T_{i,j}) \cap E(T_{i',j'}) = \emptyset$. Son independientes.
- $S_i \parallel S_k$: comparten únicamente lecturas de M por filas distintas y escriben variables distintas ($P[i]$ y $P[k]$), por lo que las tres condiciones se cumplen. Son independientes.
- **Fase 1 $\rightarrow$ Fase 2 $\rightarrow$ JOIN:** existen dependencias RAW (Fase 2 lee lo que Fase 1 escribe; JOIN lee lo que Fase 2 escribe). Deben respetar ese orden. Escritura directa $C[k++] \leftarrow a_i$ violaría Bernstein porque el índice k sería escrito por todos.

Grafo de dependencias

El grafo tiene tres niveles: $n \cdot m$ nodos iniciales en paralelo, una capa de reducción y un punto de unión final.



El camino crítico es $T_{i,j} \rightarrow S_i \rightarrow \text{JOIN}$ y el paralelismo máximo es $n \cdot m$ en la Fase 1 ($T_{i,j}$ entre sí, y S_i entre sí).

Solución con COBEGIN–COEND

```

1 leer A[1..n], B[1..m]
2 COBEGIN
3     // Fase 1: comparaciones T_{i,j} en paralelo
4     M[i,j] = (A[i] == B[j])
5 COEND
6 COBEGIN
7     // Fase 2: reduccion OR por filas S_i en paralelo
8     P[i] = OR_j M[i,j]
9 COEND
10 C = vacio
11 para i = 1..n hacer
12     si P[i] == 1 entonces
13         C = C union {A[i]}
14     fin si
15 fin para
16 escribir C

```

Las comparaciones $T_{i,j}$ son independientes entre sí, por lo que van en el primer **COBEGIN**. La reducción S_i es el punto de unión por filas y va en el segundo **COBEGIN**. La compactación de C permanece secuencial porque todas las escrituras dependerían del mismo índice k . El cálculo es correctamente sincronizado por las dos barreras **COEND**. Tiempo secuencial $T_1 = O(n \cdot m \cdot L)$ y tiempo paralelo $T_p = O(\frac{nmL}{p} + \frac{nm}{p} + n)$.

6 Problema 4. Raíces reales de una ecuación cuadrática

Enunciado

Determinar las raíces reales de una ecuación cuadrática $ax^2 + bx + c = 0$ mediante la fórmula general.

Análisis del algoritmo

El cálculo se descompone de la siguiente forma:

$$D = b^2 - 4ac, \quad x_1 = \frac{-b + \sqrt{D}}{2a}, \quad x_2 = \frac{-b - \sqrt{D}}{2a}.$$

Si $D < 0$ no existen raíces reales y se detiene el cálculo.

Condiciones de Bernstein

Para cada tarea se identifican las variables que lee (L) y las que escribe (E):

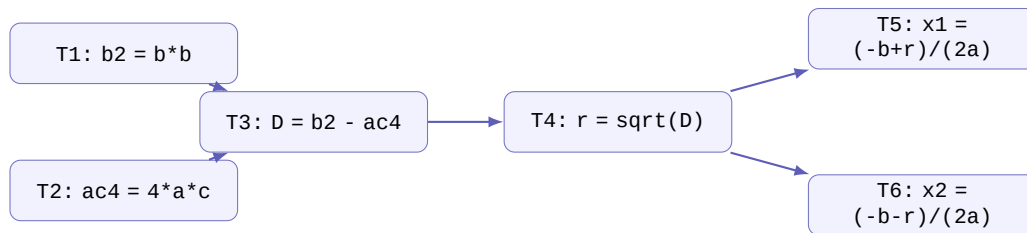
Tarea	Lee L	Escribe E
T1: $b^2 = b * b$	$\{b\}$	$\{b^2\}$
T2: $ac^4 = 4 * a * c$	$\{a, c\}$	$\{ac^4\}$
T3: $D = b^2 - ac^4$	$\{b^2, ac^4\}$	$\{D\}$
T4: $r = \text{sqrt}(D)$	$\{D\}$	$\{r\}$
T5: $x_1 = (-b+r)/(2a)$	$\{a, b, r\}$	$\{x_1\}$
T6: $x_2 = (-b-r)/(2a)$	$\{a, b, r\}$	$\{x_2\}$

Verificación de independencia:

- **T1 || T2:** $L(T1) \cap E(T2) = \emptyset$, $E(T1) \cap L(T2) = \emptyset$ y $E(T1) \cap E(T2) = \emptyset$. Son independientes.
- **T5 || T6:** comparten únicamente *lecturas* de $\{a, b, r\}$ y escriben variables distintas (x_1 y x_2), por lo que las tres condiciones se cumplen. Son independientes.
- **T1, T2 → T3 → T4:** existen dependencias RAW (T3 lee lo que T1 y T2 escriben; T4 lee lo que T3 escribe). Deben respetar ese orden.

Grafo de dependencias

El grafo tiene forma de “diamante”: dos tareas iniciales en paralelo, un punto de unión y luego dos tareas finales en paralelo.



El camino crítico es $T1 \rightarrow T3 \rightarrow T4 \rightarrow T5$ y el paralelismo máximo es 2 (T1 con T2, y T5 con T6).

Solución con COBEGIN–COEND

```

1 leer a, b, c
2 COBEGIN
3     b2 = b * b
4     ac4 = 4 * a * c
5 COEND
6 D = b2 - ac4
7 si D < 0 entonces
8     escribir "No hay raíces reales"
9 sino
10    r = sqrt(D)
11    COBEGIN
12        x1 = (-b + r) / (2*a)
13        x2 = (-b - r) / (2*a)
14    COEND
15    escribir x1, x2
16 fin si
  
```

Los dos primeros cálculos son independientes entre sí, por lo que van en el primer COBEGIN. El cálculo de D es el punto de unión (join) obligatorio. Una vez obtenida r , las dos raíces se calculan en paralelo en el segundo COBEGIN. La decisión sobre $D < 0$ permanece secuencial porque todas las tareas posteriores dependen de ella.

7 Problema 5. Máximo de una matriz

Enunciado

Dado un arreglo bidimensional A de tamaño $N \times M$ (N filas y M columnas), se requiere diseñar un algoritmo concurrente que determine el valor máximo global de la matriz utilizando paralelismo de datos, modelado con grafos de dependencia, condiciones de Bernstein y primitivas COBEGIN - COEND.

Análisis del algoritmo

La matriz se particiona horizontalmente asignando una fila completa a cada proceso concurrente:

1. **Fase Concurrente** ($S_0, S_1, \dots, S_{N-1}$): Cada proceso S_i calcula el valor máximo de su fila correspondiente y lo almacena en la posición i de un vector auxiliar unidimensional max_fila :

$$\text{max_fila}[i] = \max_{0 \leq j < M} \{A[i][j]\}.$$

2. **Fase de Reducción Secuencial** (S_{final}): Un proceso final recorre el arreglo auxiliar max_fila de longitud N y extrae el máximo global:

$$\text{max_global} = \max_{0 \leq i < N} \{\text{max_fila}[i]\}.$$

Condiciones de Bernstein

Para cada proceso se identifican los conjuntos de variables que lee (L) y que escribe (E):

Proceso / Tarea	Lee L	Escribe E
S_i (fila i , con $0 \leq i < N$)	$\{A[i][j] \mid 0 \leq j < M\}$	$\{\text{max_fila}[i]\}$
S_{final} (reducción)	$\{\text{max_fila}[i] \mid 0 \leq i < N\}$	$\{\text{max_global}\}$

Verificación de independencia:

- **Entre filas independientes** ($S_a \parallel S_b$, con $a \neq b$):

$$L(S_a) \cap E(S_b) = \emptyset, \quad E(S_a) \cap L(S_b) = \emptyset, \quad E(S_a) \cap E(S_b) = \emptyset.$$

Cada proceso lee elementos de su propia fila y escribe en una posición desacoplada del arreglo. Al cumplirse simultáneamente las tres condiciones, todos los procesos $\{S_0, S_1, \dots, S_{N-1}\}$ pueden ejecutarse en paralelo.

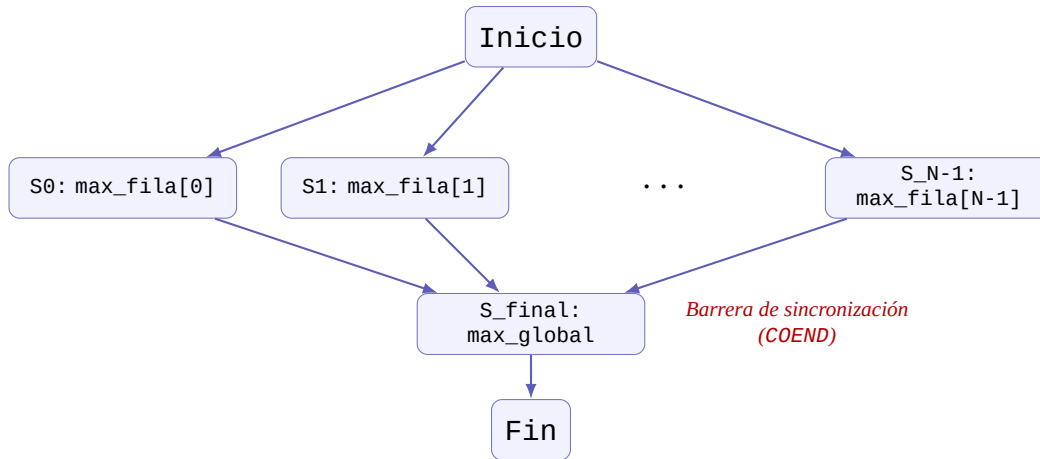
- **Dependencia con el proceso final** ($S_i \rightarrow S_{\text{final}}$):

$$E(S_i) \cap L(S_{\text{final}}) = \{\text{max_fila}[i]\} \neq \emptyset.$$

Existe una dependencia estricta de tipo RAW (*Read-After-Write*). Por ende, S_{final} no puede ejecutarse concurrentemente con las filas y requiere una barrera de sincronización previa.

Grafo de dependencias

Los procesos de fila se bifurcan en paralelo y convergen mediante una barrera de sincronización (COEND) antes de ejecutar la reducción secuencial:



El camino crítico es Inicio $\rightarrow S_i \rightarrow S_{\text{final}} \rightarrow \text{Fin}$ y el paralelismo máximo es N .

Solución con COBEGIN–COEND

```

1 // Entrada: A[N][M] (Matriz de NxM)
2 // Auxiliar: max_fila[N]
3 // Salida: max_global
4
5 COBEGIN
6     S_0;      // max_fila[0]  = max(A[0][0..M-1])
7     S_1;      // max_fila[1]  = max(A[1][0..M-1])
8     ...
9     S_{N-1};  // max_fila[N-1] = max(A[N-1][0..M-1])
10 COEND
11
12 S_final;     // max_global    = max(max_fila[0..N-1])
13 escribir max_global

```

La búsqueda del máximo en cada fila es totalmente independiente, por lo que se ejecuta dentro del bloque **COBEGIN**. La variable `max_global` representa la consolidación de los resultados intermedios y sólo puede calcularse una vez que la barrera **COEND** garantiza que todas las filas concluyeron su trabajo.

8 Problema 6. Alumnos aprobados en matemáticas

Enunciado

Mostrar el nombre de los alumnos que han aprobado el curso de matemáticas. Para aprobar, *cada* examen parcial debe tener calificación mayor o igual que 6. Cada estudiante realizó 3 exámenes parciales que valen el 50% de su calificación y 4 actividades que valen el otro 50%. De cada estudiante se conoce el nombre, la edad y las calificaciones.

Análisis del algoritmo

Para cada estudiante i se calcula:

$$pp_i = \frac{p_{1i} + p_{2i} + p_{3i}}{3}, \quad pa_i = \frac{a_{1i} + a_{2i} + a_{3i} + a_{4i}}{4},$$

$$cf_i = 0.5 pp_i + 0.5 pa_i,$$

$$\text{aprobado}_i = (p_{1i} \geq 6) \wedge (p_{2i} \geq 6) \wedge (p_{3i} \geq 6) \wedge (cf_i \geq 6).$$

Condiciones de Bernstein

El punto clave es que **los estudiantes son independientes entre sí**. Para dos estudiantes $i \neq j$:

$$L(P_i) = \{p_{1i}, p_{2i}, p_{3i}, a_{1i}, \dots, a_{4i}\}, \quad E(P_i) = \{pp_i, pa_i, cf_i, \text{aprobado}_i\},$$

y sus conjuntos son disjuntos de los del estudiante j . Por lo tanto

$$L(P_i) \cap E(P_j) = \emptyset, \quad E(P_i) \cap L(P_j) = \emptyset, \quad E(P_i) \cap E(P_j) = \emptyset,$$

es decir, todos los estudiantes pueden procesarse en paralelo (paralelismo de datos).

Dentro de cada estudiante, las tareas son:

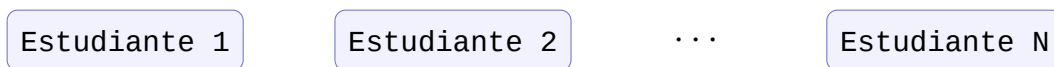
Tarea	Lee L	Escribe E
T1: $pp = (p_1 + p_2 + p_3)/3$	$\{p_1, p_2, p_3\}$	$\{pp\}$
T2: $pa = (a_1 + a_2 + a_3 + a_4)/4$	$\{a_1, a_2, a_3, a_4\}$	$\{pa\}$
T3: $cf = 0.5 * pp + 0.5 * pa$	$\{pp, pa\}$	$\{cf\}$
T4: $\text{aprobado} = (p_1 \geq 6) \text{ y } \dots \text{ y } (cf \geq 6)$	$\{p_1, p_2, p_3, cf\}$	$\{\text{aprobado}\}$
T5: imprimir nombre si aprobado	$\{\text{aprobado}, \text{nombre}\}$	$\{\}$

T1 || **T2**: escriben variables distintas y leen datos distintos, por lo que las tres condiciones de Bernstein se cumplen. **T3** es el punto de unión (depende de pp y pa), **T4** depende de cf y **T5** de aprobado.

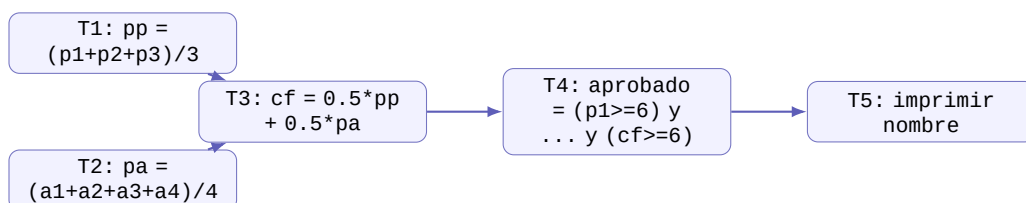
Grafo de dependencias

Entre estudiantes no existe ninguna arista: el grafo queda formado por N componentes independientes (paralelismo ideal). Dentro de cada estudiante sí hay dependencias:

Entre estudiantes: sin aristas (N componentes independientes).



Dentro de cada estudiante:



Solución con COBEGIN–COEND

```
1 COBEGIN // un proceso por estudiante
2   para i = 1 .. N
3     COBEGIN
4       pp[i] = (p1[i] + p2[i] + p3[i]) / 3
5       pa[i] = (a1[i] + a2[i] + a3[i] + a4[i]) / 4
6     COEND
7     cf[i] = 0.5*pp[i] + 0.5*pa[i]
8     aprobado[i] = (p1[i]>=6) y (p2[i]>=6) y (p3[i]>=6) y (cf[i]>=6)
9   COEND
10
11 para i = 1 .. N // impresion secuencial
12   si aprobado[i] entonces escribir nombre[i]
```

La impresión es un **recurso compartido** (sección crítica): si se realizara en paralelo habría que protegerla con un mutex para evitar salida entrelazada; por eso se hace de forma secuencial. El cálculo, en cambio, es totalmente paralelizable porque cada estudiante trabaja sobre sus propias variables.

9 Problema 7. Descuento en medicamentos

Enunciado

Se requiere conocer si una persona puede obtener un descuento en la compra de sus medicamentos. Una persona puede obtener un descuento del 20% sobre su compra total en la compra de al menos \$1000.00, o un descuento del 30% en la compra de \$1500, o un descuento del 10% en la compra de al menos un antiácido de cualquier costo. Los descuentos no son acumulables en la misma compra.

Análisis del algoritmo

Sean T el total de la compra y q el número de antiácidos comprados. Las tres condiciones se evalúan de forma independiente:

$$c_{20} = (T \geq 1000), \quad c_{30} = (T \geq 1500), \quad c_{10} = (q \geq 1).$$

Como los descuentos no son acumulables, se aplica uno solo, eligiendo el de mayor beneficio:

$$d = \begin{cases} 30\% & c_{30} = 1, \\ 20\% & c_{30} = 0 \wedge c_{20} = 1, \\ 10\% & c_{30} = 0 \wedge c_{20} = 0 \wedge c_{10} = 1, \\ 0\% & \text{en otro caso.} \end{cases}$$

El total a pagar es $T - d \cdot T$.

Condiciones de Bernstein

Para cada tarea se identifican las variables que lee (L) y las que escribe (E):

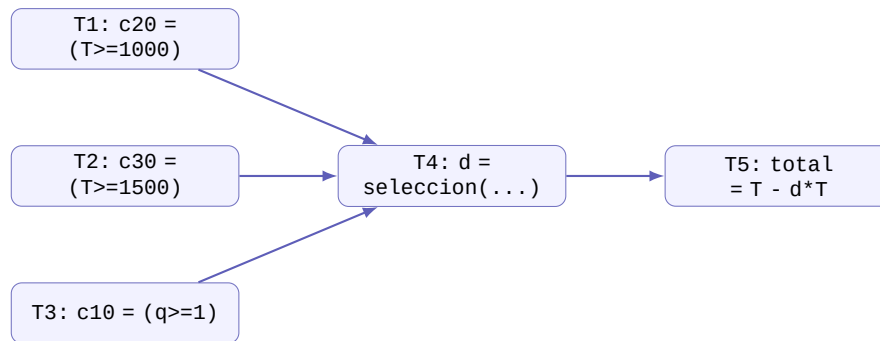
Tarea	Lee L	Escribe E
T1: $c_{20} = (T \geq 1000)$	$\{T\}$	$\{c_{20}\}$
T2: $c_{30} = (T \geq 1500)$	$\{T\}$	$\{c_{30}\}$
T3: $c_{10} = (q \geq 1)$	$\{q\}$	$\{c_{10}\}$
T4: $d = \text{seleccion}(c_{20}, c_{30}, c_{10})$	$\{c_{20}, c_{30}, c_{10}\}$	$\{d\}$
T5: $\text{total} = T - d * T$	$\{T, d\}$	$\{\text{total}\}$

Verificación de independencia:

- **T1 || T2 || T3:** comparten únicamente *lecturas* (T o q) y escriben variables distintas (c_{20} , c_{30} y c_{10}), por lo que las tres condiciones se cumplen. Son independientes.
- **T1, T2, T3 → T4 → T5:** existen dependencias RAW (T4 lee lo que T1, T2 y T3 escriben; T5 lee lo que T4 escribe). Deben respetar ese orden.

Grafo de dependencias

Las tres condiciones se evalúan en paralelo y convergen en la selección del descuento, que es el punto de unión obligatorio.



El camino crítico es $T1 \rightarrow T4 \rightarrow T5$ y el paralelismo máximo es 3 (T1, T2 y T3).

Solución con COBEGIN–COEND

```

1 leer T, q
2 COBEGIN
3   c20 = (T >= 1000)
4   c30 = (T >= 1500)
5   c10 = (q >= 1)
6 COEND
7 si c30 = 1 entonces
8   d = 0.30
  
```

```
9  sino si c20 = 1 entonces
10      d = 0.20
11  sino si c10 = 1 entonces
12      d = 0.10
13  sino
14      d = 0
15  fin si
16  total = T - d * T
17  escribir total
```

Las tres condiciones se calculan en paralelo porque sólo leen datos y escriben variables distintas, por lo que van en el mismo **COBEGIN**. La selección del descuento es el punto de unión (join) obligatorio y permanece secuencial, ya que depende de los tres resultados y los descuentos no son acumulables.

10 Conclusiones

El modelado paralelo se apoya en el análisis de dependencias y no en la intuición. Aplicando las condiciones de Bernstein se decide qué bloques son independientes y pueden ir dentro de un mismo **COBEGIN**, mientras que el grafo de dependencias muestra el orden obligatorio y el paralelismo real disponible. Los ejercicios con datos independientes por elemento (como el problema 6) alcanzan un paralelismo casi perfecto, mientras que los que tienen una cadena de cálculo común (como el problema 4) quedan limitados por su camino crítico.